

INFORME DE AUDITORÍA DE SEGURIDAD — ECOSISTEMA LEVEL UP

Auditor: Análisis SAST/Manual

Fecha: Junio 2026

Clasificación: Confidencial

1. SEGURIDAD EN EL BACKEND Y APIs (Node.js)

 **CRÍTICO** — Credenciales expuestas en

.env

Archivo:

D:\proyectos\level2026\.env

Hallazgo	Detalle
MongoDB URI con credenciales mongodb+srv://fixitai110980_db_user:qSifC0lNdyDkKArv@cluster0... en texto plano	
JWT Secret débil (15 chars, caracteres especiales sin escape)	BNbunZ94XWQ%M_v

Riesgo: Si este archivo llega al repositorio, se expone la base de datos completa.

Mitigación:

- Rotar inmediatamente la contraseña de MongoDB Atlas
- Usar JWT_SECRET de mínimo 64 caracteres aleatorios (
openssl rand -hex 32
)
- Verificar que

.env

esté en

.gitignore

- Usar variables de entorno del sistema en producción (no archivos)
-

● CRÍTICO — JWT con expiración excesiva

Archivo:

authController.js

Token	Expiración
Usuarios regulares	30d (30 días)
Conductores (QR)	365d (1 año)

Riesgo: Un token robado mantiene acceso durante meses. No hay mecanismo de revocación (blacklist).

Mitigación:

- Reducir expiración a

7d

para usuarios,

30d

para conductores

- Implementar refresh tokens con rotación
 - Crear blacklist de tokens en Redis/MongoDB para logout forzado
-

● ALTO — CORS configurado como wildcard (

*

)

Archivo:

server.js

—

```
app.use(cors())
```

Sin configuración explícita,

```
cors()
```

acepta **cualquier origen**. Un atacante puede crear un sitio malicioso que consuma la API autenticada del usuario.

Mitigación:

```
app.use(cors({  
  origin: ['http://localhost:3000', 'http://localhost:5173', 'https://tu-dominio.com'],  
  credentials: true,  
}));
```

Copyjavascript

● ALTO — Sin Rate Limiting (Fuerza Bruta)

No hay protección contra ataques de fuerza bruta en

/api/auth/login

. Un atacante puede probar miles de contraseñas por minuto.

Mitigación:

- Instalar

express-rate-limit

:

```
import rateLimit from 'express-rate-limit';
```

```
app.use('/api/auth', rateLimit({ windowMs: 15*60*1000, max: 10 }));
```

● ALTO — Sin validación de entrada en registro/perfil

El endpoint

POST /api/auth/register

acepta directamente

req.body

sin sanitizar. Un atacante puede inyectar campos como

role: "admin"

para escalamiento de privilegios.

Archivo:

authController.js

línea ~18:

role

se lee directamente del body.

Mitigación:

- Nunca aceptar

role

desde el cliente en registro. Forzar

role: 'client'

- Implementar validación con

joi

o

express-validator

● MEDIO — Base64 sin límite de tamaño real

El

```
express.json({ limit: '10mb' })
```

permite payloads de hasta 10MB. Las fotos en Base64 y

fotoUrl

no validan tamaño ni tipo MIME. Un atacante puede enviar payloads pesados para DoS.

Mitigación:

- Validar que Base64 sea imagen real (verificar magic bytes)
- Limitar

fotoUrl

a 2MB máximo en el esquema de Mongoose

MEDIO — Endpoint

/api/access/validate/:token

sin rate limit

Es público (diseñado para Raspberry Pi) pero sin protección. Un atacante puede hacer brute force de tokens.

Mitigación:

- Rate limit agresivo: 30 req/min por IP
- Tokens de 64 hex chars hacen brute force impracticable (2^{128} combinaciones), pero el rate limit previene abuso de red

BAJO — Helmet configurado correctamente ✓

helmet()

está presente y activo, proveyendo headers de seguridad estándar.

2. SEGURIDAD MOBILE (Flutter — Alumnos y Choferes)

✓ BUENA PRÁCTICA —

`flutter_secure_storage`

para JWT

Ambas apps (alumnos y conductores) usan

`FlutterSecureStorage`

correctamente para almacenar tokens JWT. No se usa

`SharedPreferences`

para datos sensibles.

● ALTO — API Keys de Google Maps hardcodeadas

Archivos:

- `AndroidManifest.xml`

:

`AlzaSyD3m_c376fKrnVQzyWQ5oa5mUk0yfKFjww`

- `AppDelegate.swift`

:

`AlzaSyDLsWInyAt_nWNy3tPDIdGhjz3DoIdQM90`

Riesgo: Un decompilador puede extraer estas keys y usarlas para generar costos en tu cuenta Google Cloud.

Mitigación:

- Restringir keys en Google Cloud Console por package name (

`com.jorge.level2026`

) y SHA-1

- Habilitar cuotas diarias en la consola de GCP
- Usar

`--obfuscate --split-debug-info`

al compilar en release

● ALTO — Llave pública Wompi hardcodeda en Dart

Archivo:

cart_screen.dart

```
static const _wompiPublicKey = 'pub_stagtest_g2u0HQd3ZMh05hsSgTS2IUv8t3s4mOt7';
```

Copydart

Riesgo: Es una llave pública de sandbox (no es secreta), pero en producción debe obtenerse del backend, no hardcodearse.

Mitigación:

- En producción, el backend debe generar la URL de checkout y enviarla al cliente
- Nunca poner

prv_

(llave privada) en código cliente

● ALTO — Transmisión GPS sin validación de integridad

Archivo:

gps_service.dart

(conductores)

Las coordenadas se envían via HTTP al backend sin ninguna validación de que provienen de un GPS real. Un conductor malicioso con

MockGPS

puede falsificar su ubicación.

Mitigación:

- Implementar detección de mock location en Android (

Position.isMocked

)

- Validar en backend que las coordenadas no salten distancias imposibles entre pings (>200km en 5s)
 - Agregar timestamp firmado con HMAC
-

● MEDIO — Comunicación HTTP en desarrollo

Las apps usan

`http://10.0.2.2:5000`

y

`http://192.168.x.x:5000`

. En producción DEBE ser HTTPS.

Mitigación:

- Forzar HTTPS en producción con certificate pinning
- Android: configurar

`network_security_config.xml`

para bloquear cleartext

- iOS: App Transport Security (ATS) ya bloquea HTTP por defecto
-

● MEDIO — Sin ofuscación de código

El código Dart se compila a binario pero sin ofuscación. Un atacante puede usar herramientas como

`jadx`

o

`flutter-tools`

para inspeccionar la lógica.

Mitigación:

- Compilar releases con:

`flutter build apk --obfuscate --split-debug-info=./debug-info`

3. SEGURIDAD EN EL PANEL WEB (React)

● ALTO — Sin verificación de rol en `ProtectedRoute`

Archivo:

`ProtectedRoute.jsx`

Solo valida que

`isAuthenticated`

sea

`true`

. No verifica que el usuario sea admin. Un alumno (role:

`client`

) con un token válido podría acceder a

`/dashboard`

,

`/users`

,

`/classes`

del panel si conoce las URLs.

Mitigación:

`if (!isAuthenticated || user?.role !== 'admin') return <Navigate to="/login" />;`

Copyjsx

⚠ **Sin embargo**, la protección real está en el backend (

`authorize('admin')`

) en los endpoints de escritura. Pero los endpoints de lectura como

GET /api/classes

y

GET /api/teachers

son accesibles por cualquier usuario autenticado.

● MEDIO — XSS via Base64 en imágenes

Las fotos se renderizan como

```

```

. Si un atacante inyecta un payload SVG con JavaScript como

fotoUrl

, podría ejecutar scripts.

Mitigación:

- Validar en backend que el Base64 comience con magic bytes de JPEG/PNG
- Usar

Content-Security-Policy: img-src 'self' data:

via Helmet

- Sanitizar

fotoUrl

antes de almacenar

● MEDIO — Token almacenado en localStorage

Archivo:

AuthContext.jsx

—

```
localStorage.setItem('token', tkn)
```

localStorage

es accesible via XSS. Si hay una vulnerabilidad XSS, el token se puede robar.

Mitigación:

- Usar

httpOnly cookies

para el token (requiere cambio en backend)

- O mantener localStorage pero garantizar 0 XSS con CSP estricto

4. RESUMEN Y CLASIFICACIÓN

#	Vulnerabilidad	Severidad	Componente
1	Credenciales MongoDB en .env sin rotación	 Crítico	Backend
2	JWT Secret débil + expiración 30d sin revocación	 Crítico	Backend
3	Rol admin aceptado desde registro del cliente	 Alto	Backend
4	CORS wildcard (*)	 Alto	Backend
5	Sin rate limiting en login	 Alto	Backend

#	Vulnerabilidad	Severidad	Componente
6	API Keys Google Maps expuestas sin restricción	 Alto	Mobile
7	GPS sin validación anti-spoofing	 Alto	Mobile (Choferes)
8	Panel admin sin verificación de rol en frontend	 Alto	Panel Web
9	Base64 sin validación de tipo MIME	 Medio	Backend
10	Token validate sin rate limit	 Medio	Backend
11	HTTP en dev (sin HTTPS)	 Medio	Todos
12	Sin ofuscación de código	 Medio	Mobile
13	XSS potencial via Base64 SVG	 Medio	Panel Web
14	Token en localStorage	 Medio	Panel Web
15	Helmet configurado ✓	 OK	Backend
16	flutter_secure_storage ✓	 OK	Mobile
17	Bcrypt con 12 rounds ✓	 OK	Backend
18	Password	 OK	Backend

#	Vulnerabilidad	Severidad	Componente
	select: false		
	en schema ✓		

5. ACCIONES PRIORITARIAS (Top 5)

1. **Rotar JWT_SECRET** a 64+ chars y reducir expiración a 7d

2. **Bloquear**

role

en registro — forzar

'client'

siempre

3. **Configurar CORS** con whitelist de orígenes específicos

4. **Agregar**

express-rate-limit

en

/api/auth/login

y

/api/access/validate

5. **Restringir API Keys** de Google Maps por package name + SHA-1 en GCP